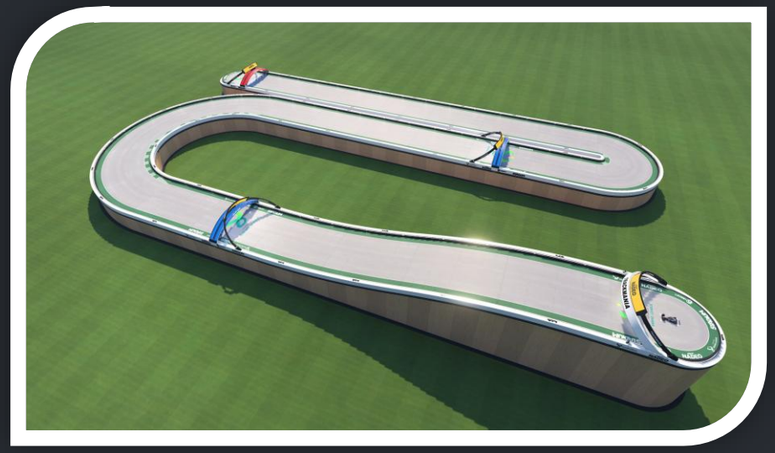


Autonomous Driving Project (Trackmania)

Deep learning model for Trackmania autonomous driving.

Goal: Navigate vehicle using directional commands in real-time gameplay.

Katoo, Ege, Francesco

Data Capture and Logging

1

Custom Script

Captured frames and key presses to simulate human driving.

2

Dataset Expansion

Iteratively expanded dataset over multiple collection rounds.

3

Turn Underrepresentation

Impacting real-world model performance.

```
timestamp,frame_path,key,speed
1729690862651,./data/frames/up\frame_1729690862651.jpg,up,95.38286051866291
1729690862842,./data/frames/up\frame_1729690862842.jpg,up,105.90778645371171
1729690863030,./data/frames/up\frame_1729690863030.jpg,up,190.7494112082615
1729690863229,./data/frames/up\frame_1729690863229.jpg,up,152.1780934469571
1729690863465,./data/frames/up\frame_1729690863465.jpg,up,52.71616658725164
1729690863659,./data/frames/up\frame_1729690863659.jpg,up,1.8420844154319216
1729690863855,./data/frames/up\frame_1729690863855.jpg,up,138.73197355024067
1729690864049,./data/frames/up\frame_1729690864049.jpg,up,161.7366176221537
1729690864261,./data/frames/up\frame_1729690864261.jpg,up,152.92139192363808
1729690864475,./data/frames/up\frame_1729690864475.jpg,up,39.9331015397189
1729690864665,./data/frames/up\frame_1729690864665.jpg,up,195.72285767747397
1729690864862,./data/frames/up\frame_1729690864862.jpg,up,80.71302370637024
1729690865065,./data/frames/up\frame_1729690865065.jpg,up,146.7088883618366
1729690865261,./data/frames/up\frame_1729690865261.jpg,up,74.93053942375853
1729690865456,./data/frames/up\frame_1729690865456.jpg,up,186.53623311333536
1729690865633,./data/frames/up\frame_1729690865633.jpg,up,134.42823723629388
1729690906967,./data/frames/right\frame_1729690906967.jpg,right,101.1144628888329
1729690907196,./data/frames/right\frame_1729690907196.jpg,right,33.50357269999649
1729690907401,./data/frames/right\frame_1729690907401.jpg,right,178.9458663432811
1729690907629,./data/frames/right\frame_1729690907629.jpg,right,24.2629521804989
1729690907873,./data/frames/right\frame_1729690907873.jpg,right,173.36449303819782
1729690908079,./data/frames/right\frame_1729690908079.jpg,right,160.43438210395723
1729690908271,./data/frames/right\frame_1729690908271.jpg,right,182.9309737048949
1729690908482,./data/frames/right\frame_1729690908482.jpg,right,7.887857724496783
1729690908687,./data/frames/right\frame_1729690908687.jpg,right,141.29570429145562
1729690908884,./data/frames/right\frame_1729690908884.jpg,right,126.25682697408813
1729690909089,./data/frames/right\frame_1729690909089.jpg,right,96.63880269155992
1729690909312,./data/frames/right\frame_1729690909312.jpg,right,134.3027188755809
1729690909520,./data/frames/right\frame_1729690909520.jpg,right,140.10535658836758
1729690909721,./data/frames/right\frame_1729690909721.jpg,right,172.19551399245194
1729690909914,./data/frames/right\frame_1729690909914.jpg,right,171.13618416594235
1729690910111,./data/frames/right\frame_1729690910111.jpg,right,170.02309379499675
1729690910308,./data/frames/right\frame_1729690910308.jpg,right,170.3266286364054
```

Data Augmentation and Balancing

Augmentation Techniques

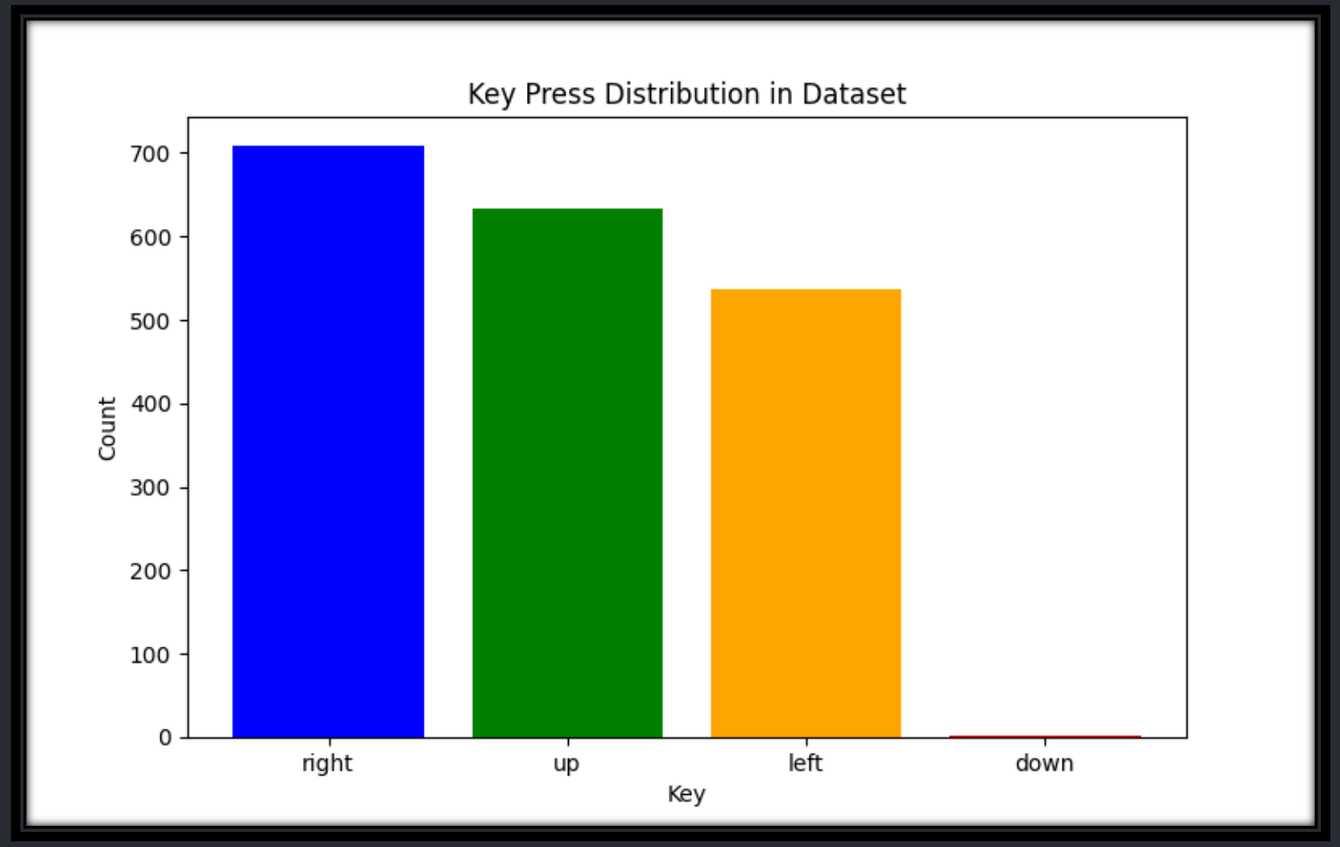
Employed to expand dataset and improve generalization.

Balance Issues

Data imbalance persisted, affecting model performance.

Turn Data

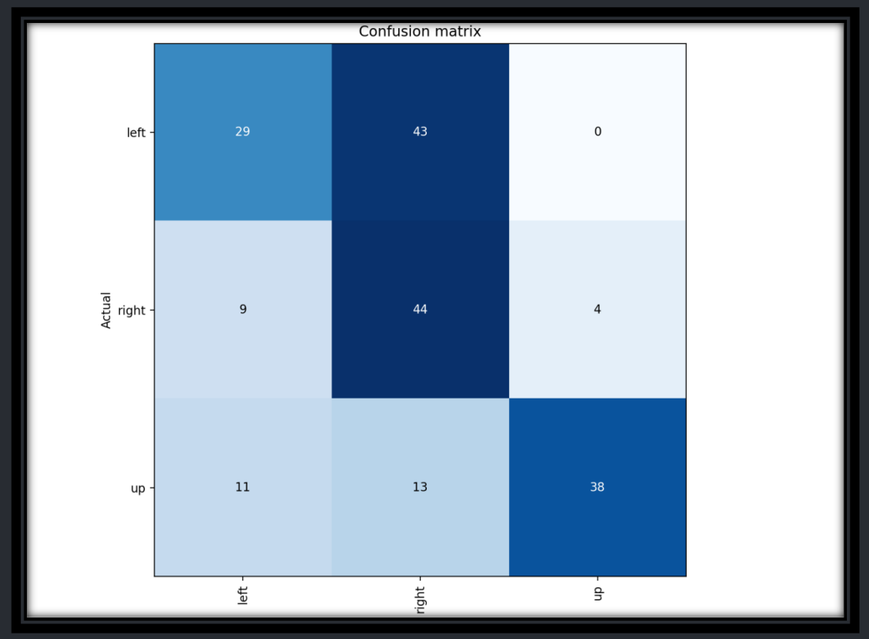
Focus on increasing representation of turn scenarios.



Initial Models: ResNet-18 and ResNet-34

Non-Pretrained ResNet-18

Baseline with 65-70% accuracy, struggled with turns.



Pretrained ResNet-18 and ResNet-34

Improved straight paths, issues in turn interpretation remained.

43	1.205296	1.820289	0.502618	0.662195	0.502618	0.412489	00:20
44	1.171612	1.173115	0.392670	0.750785	0.392670	0.310319	00:21
45	1.231643	1.119049	0.596859	0.714682	0.596859	0.587070	00:21
46	1.201408	0.955112	0.497382	0.486163	0.497382	0.401848	00:20
47	1.168483	1.092821	0.507853	0.489100	0.507853	0.415597	00:20
48	1.171661	1.040687	0.455497	0.412095	0.455497	0.371984	00:20
49	1.130199	1.005501	0.445026	0.417589	0.445026	0.357965	00:20
50	1.142553	53.293575	0.366492	0.330137	0.366492	0.295927	00:20
51	1.130187	0.994053	0.413613	0.425261	0.413613	0.320568	00:20
52	1.126710	1.157487	0.382199	0.441900	0.382199	0.339300	00:20
53	1.113979	1.046749	0.497382	0.571614	0.497382	0.415763	00:20
54	1.114820	2.826762	0.476440	0.645689	0.476440	0.436501	00:20
55	1.090613	0.887932	0.586387	0.469032	0.586387	0.492019	00:20
56	1.075073	14.356949	0.502618	0.533662	0.502618	0.437956	00:20
57	1.077769	1.074280	0.602094	0.478204	0.602094	0.508740	00:20
58	1.054815	37.331177	0.523560	0.570477	0.523560	0.532013	00:20
59	1.054993	54.269035	0.612565	0.623511	0.612565	0.616124	00:20
60	1.064122	5.650948	0.539267	0.593082	0.539267	0.555084	00:20
61	1.043421	9.714031	0.591623	0.650845	0.591623	0.585499	00:20
62	1.030021	60.946678	0.534031	0.628712	0.534031	0.499753	00:20
63	0.995777	17.102800	0.539267	0.668237	0.539267	0.530414	00:20
64	1.014731	26.152924	0.492147	0.556704	0.492147	0.409570	00:20
65	1.011947	9.300564	0.476440	0.399845	0.476440	0.390774	00:20
66	0.996323	19.266073	0.476440	0.399845	0.476440	0.390774	00:20
67	0.988888	8.549094	0.586387	0.651391	0.586387	0.593470	00:20
68	0.990608	1.239596	0.633508	0.670416	0.633508	0.640707	00:20
69	0.986879	0.882970	0.633508	0.678042	0.633508	0.640708	00:20
70	0.966533	4.153090	0.617801	0.669304	0.617801	0.625348	00:20
71	0.961270	9.316341	0.602094	0.657577	0.602094	0.608055	00:20
72	0.972260	2.541737	0.544503	0.628504	0.544503	0.520521	00:20
73	0.964523	9.407749	0.581152	0.664986	0.581152	0.576076	00:20
74	0.955103	6.412738	0.581152	0.651450	0.581152	0.585626	00:20

Dataset Restructuring

Reorganization

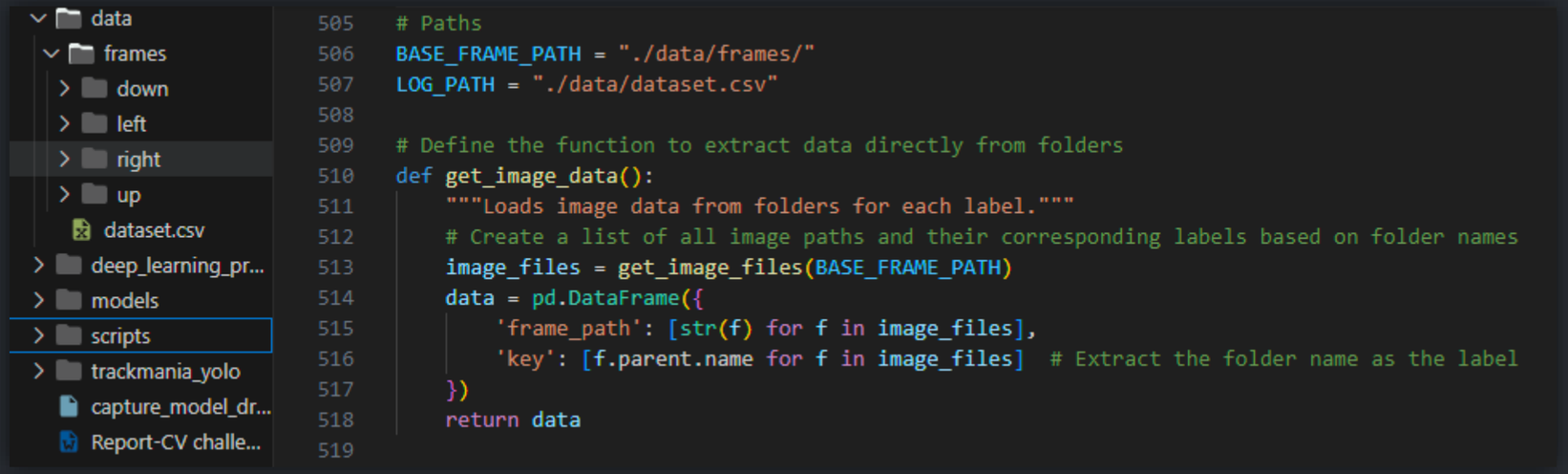
1 Restructured data for better distribution control.

Folder-Based

2 Organized dataset into structured folder system.

Retraining

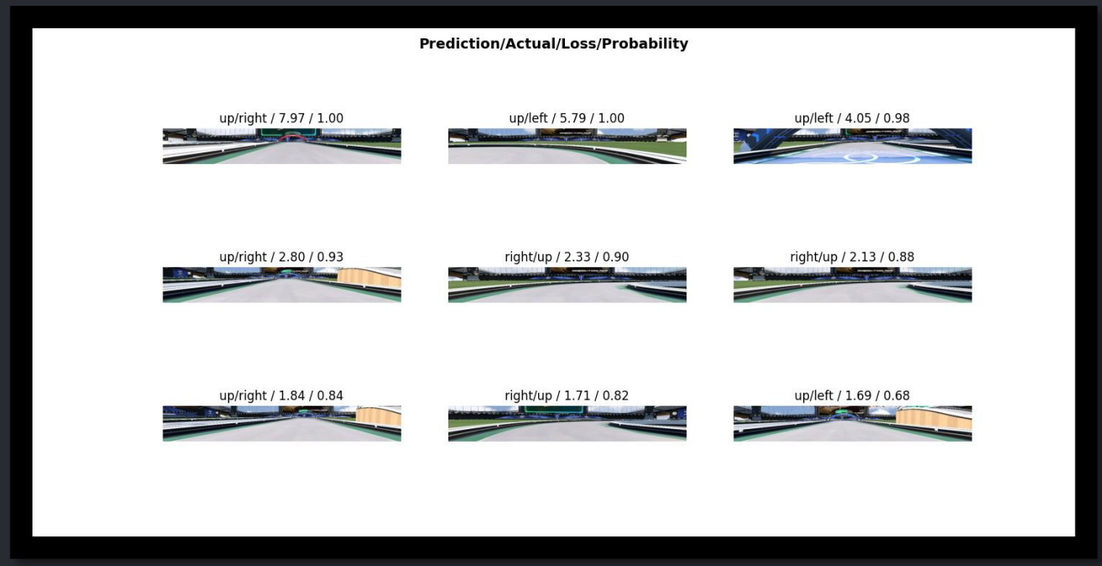
3 Models retrained on newly structured dataset.



Advanced Models: ResNet-34 and ResNet-50

Model	Pretrained	Non-Pretrained
ResNet-34	~95% accuracy	~85% accuracy
ResNet-50	~95% accuracy	~85% accuracy

Pretrained ResNet-50



Training Techniques and Optimization

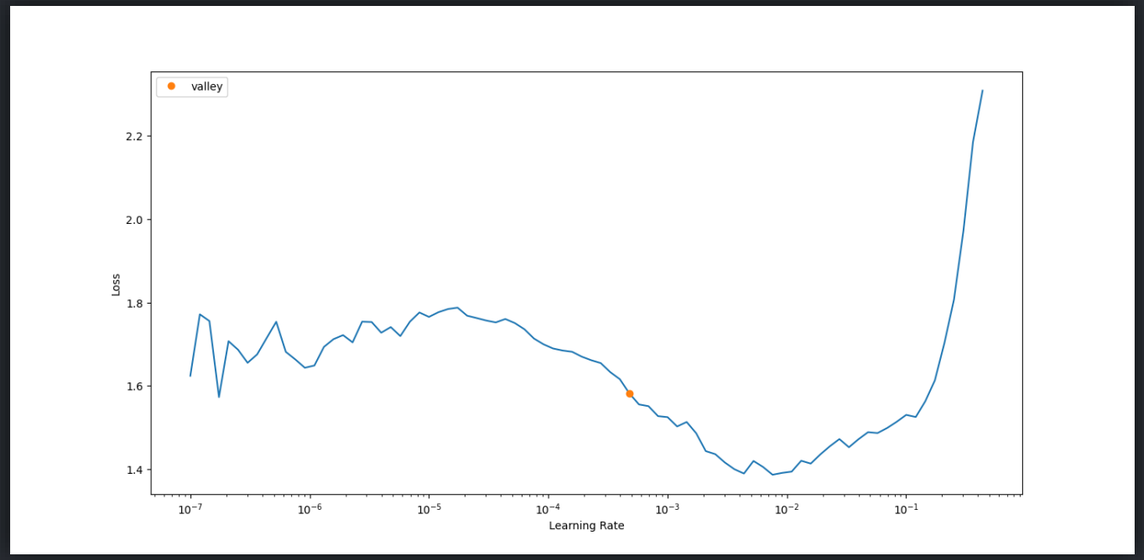
One-Cycle Policy

Used for faster convergence, limited turn prediction improvement.

```
# Define and train the model
learn = vision_learner(dls, resnet50, pretrained=True, metrics=[accuracy, Precision(average='weighted'), Recall(average='weighted'), F1Score(average='weighted')], ps=0.2)  # `ps` adds dropout

# Find the optimal learning rate
learn.lr_find()

# Unfreeze model for training the entire architecture
learn.unfreeze()

# Fine-tune the model with one-cycle learning and differential learning rates
learn.fit_one_cycle(50, lr_max=slice(1e-4, 1e-2))

# Apply mixed precision for faster training (optional)
learn = learn.to_fp16()
```

Dropout Regularization

Applied to reduce overfitting, minimal impact on turns.



Mixed Precision Training

Allowed for larger models within resource limits.

Real-Time Autonomous Driving

Capture

Real-time frame capture from Trackmania gameplay.

Process

Frame processing and action prediction.

Drive

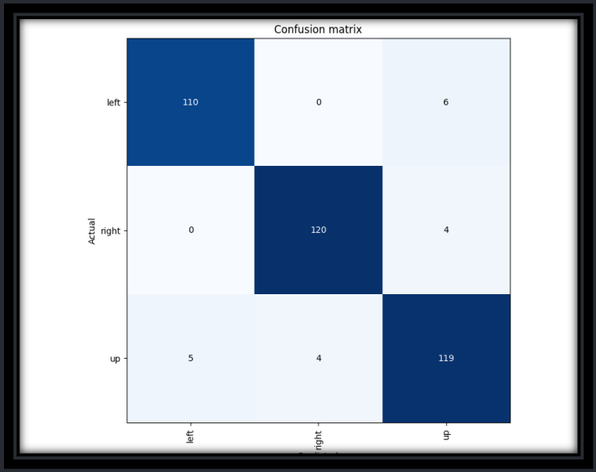
Predicted actions applied for autonomous vehicle control.

```python
# Load the trained model
model_path = './models/model_classification_fastai_v2.pkl'
learn = load_learner(model_path)
# print(learn.dls.vocab)

# Initialize keyboard controller
keyboard = Controller()

# Function to capture frames and control the car
def capture_and_drive():
    bbox = (0, 40, 1280, 720)  # Adjust based on your screen setup

    while True:
        # Capture the game screen
        screen = np.array(ImageGrab.grab(bbox=bbox))

        # Convert to RGB and then to PIL image for FastAI
        frame = cv2.cvtColor(screen, cv2.COLOR_BGR2RGB)  # Convert to RGB
        frame = PILImage.create(frame)

        # Resize to match training size
        frame = Resize(224)(frame)

        # Convert PIL image to tensor using ToTensor
        frame_tensor = ToTensor()(frame)  # Convert to tensor

        # Normalize the tensor to [0, 1] range (optional: depending on training settings)
        frame_tensor = frame_tensor.float() / 255.0  # Convert to float and normalize if required

        # Add batch dimension and move to device (GPU/CPU)
        frame_tensor = frame_tensor.unsqueeze(0).to(learn.dls.device)

        # Predict action using the loaded model
        preds = learn.model(frame_tensor)
        predicted_action = preds.argmax().item()

        # Convert index to action
        action = learn.dls.vocab[predicted_action]
        print(f"Predicted action: {action}")

        # action_vocab = ['down', 'left', 'right', 'up']
        # action = action_vocab[predicted_action]
        # print(f"Predicted action: {action}")

        # Perform the predicted action
        if action == 'left':
            keyboard.press(Key.left)
            time.sleep(0.1)
            keyboard.release(Key.left)
        elif action == 'right':
            keyboard.press(Key.right)
            time.sleep(0.1)
            keyboard.release(Key.right)
        elif action == 'up':
            keyboard.press(Key.up)
            time.sleep(0.1)
            keyboard.release(Key.up)
        elif action == 'down':
            keyboard.press(Key.down)
            time.sleep(0.1)
            keyboard.release(Key.down)
```

Performance Evaluation

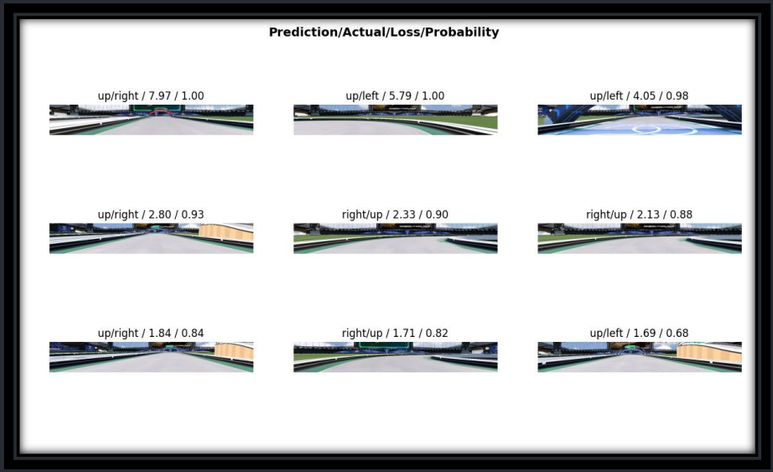
Confusion Matrix

Misclassifications of turns as straight paths.



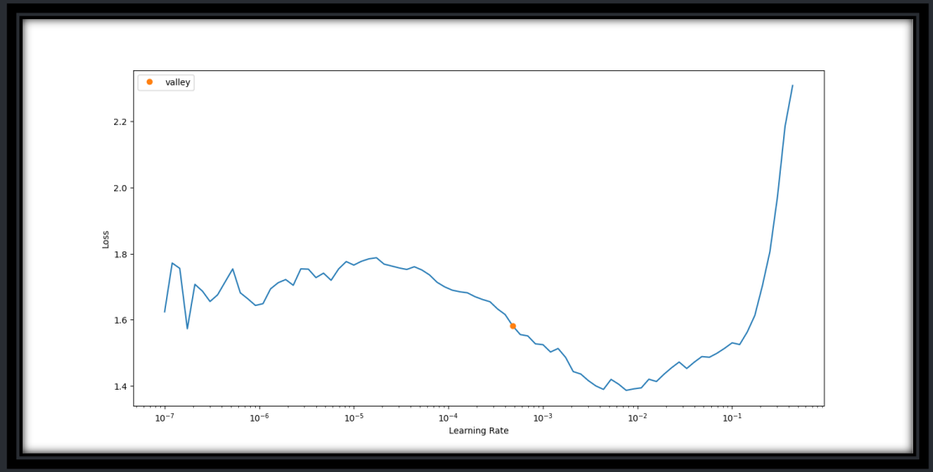
Top Loss Frames

High loss in turn scenarios.



Gameplay Testing

Real-world limitations despite high validation accuracy.



Limitations and Future Recommendations

1 **Data Imbalance**

Poor turn performance due to straight path dominance.

2 **Model Overfitting**

High test accuracy, poor real-world generalization.

3 **Temporal Modeling**

Static CNNs lack complexity for dynamic driving.

4 **Future Directions**

Explore vision transformers, reinforcement learning for improvements.

YOLO-Based Object Detection

1 Architecture

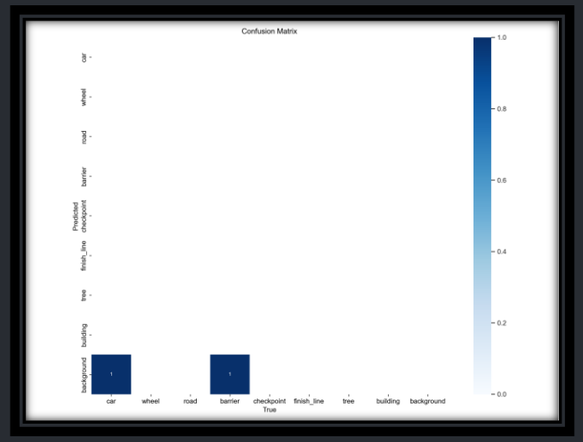
Single-pass object detection model for real-time applications.

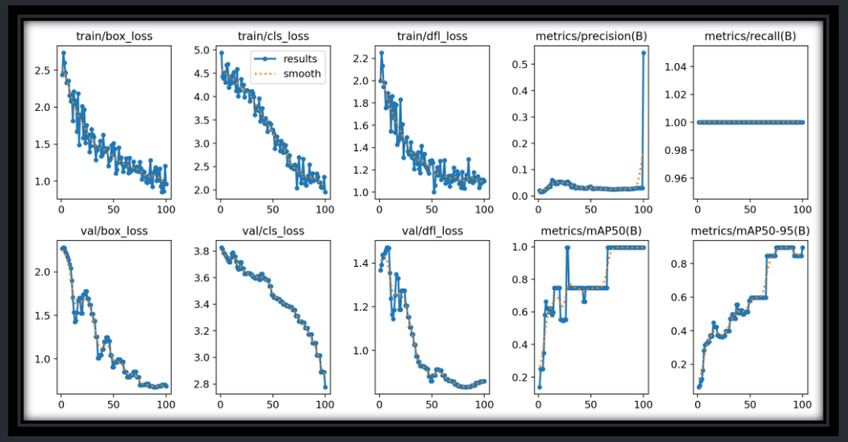
2 Process

Predicts bounding boxes and classifies objects with confidence scores.

3 Application

Adapted for Trackmania object detection tasks.





YOLO Implementation and Labeling

1

Code Workflow

Splits data into train/val sets, creates a YOLO-compatible configuration, and trains YOLOv8 on custom Trackmania dataset

2

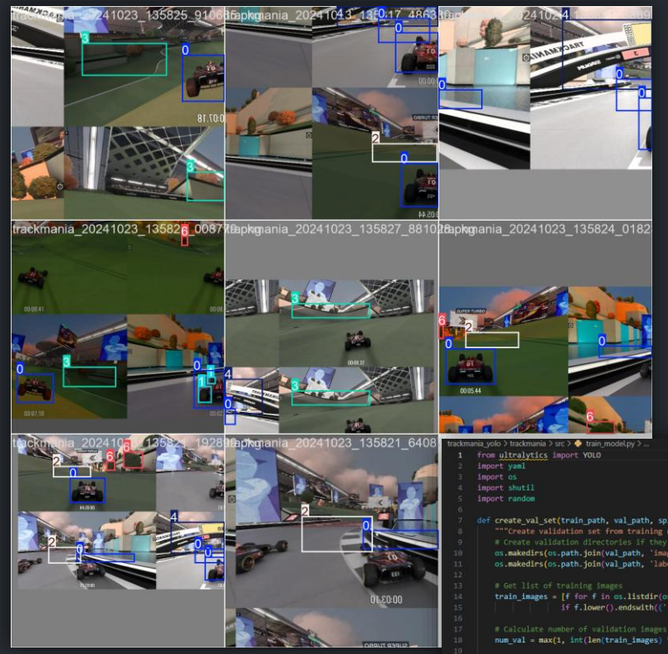
Labeling Process

Manual labeling of ~40 images to refine detection.

3

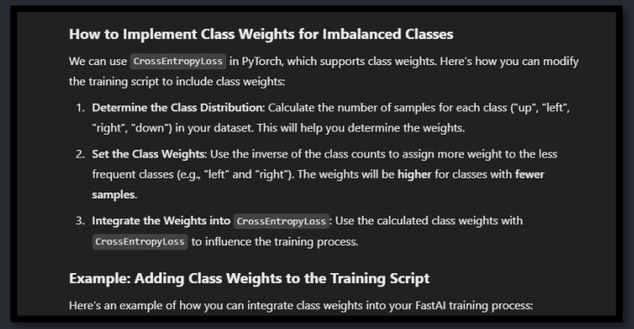
Dataset Limitation

Small dataset size limiting robust accuracy across scenarios.



```python
from ultralytics import YOLO
import yaml
import os
import shutil
import random

def create_val_set(train_path, val_path, split_ratio=0.2):
    """Create validation set from training data"""
    # Create validation directories if they don't exist
    os.makedirs(os.path.join(val_path, 'images'), exist_ok=True)
    os.makedirs(os.path.join(val_path, 'labels'), exist_ok=True)

    # Get list of training images
    train_images = [f for f in os.listdir(os.path.join(train_path, 'images'))
                    if f.lower().endswith(('.png', '.jpg', '.jpeg'))]

    # Calculate number of validation images
    num_val = max(1, int(len(train_images) * split_ratio))

    # Randomly select images for validation
    val_images = random.sample(train_images, num_val)

    # Move selected images and their labels to validation set
    for img_name in val_images:
        # Move image
        src_img = os.path.join(train_path, 'images', img_name)
        dst_img = os.path.join(val_path, 'images', img_name)
        shutil.copy2(src_img, dst_img)

        # Move corresponding label
        label_name = os.path.splitext(img_name)[0] + '.txt'
        src_label = os.path.join(train_path, 'labels', label_name)
        dst_label = os.path.join(val_path, 'labels', label_name)
        if os.path.exists(src_label):
            shutil.copy2(src_label, dst_label)

    return len(val_images)

def setup_training():
    # Get absolute path of the project directory
    project_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

    # Define Trackmania-specific classes
    classes = [
        'car',
        'wheel',
        'road',
        'barrier',
        'checkpoint',
        'finish_line',
        'tree',
        'building'
    ]

    # Create absolute paths
    dataset_path = os.path.join(project_dir, 'dataset')
    train_path = os.path.join(dataset_path, 'train')
    val_path = os.path.join(dataset_path, 'val')

    # Create train directory if it doesn't exist
    os.makedirs(os.path.join(train_path, 'images'), exist_ok=True)
    os.makedirs(os.path.join(train_path, 'labels'), exist_ok=True)

    # Verify training images exist
    train_images = [f for f in os.listdir(os.path.join(train_path, 'images'))
                    if f.lower().endswith(('.png', '.jpg', '.jpeg'))]
```

Use of Generative AI Tools



How to Implement Class Weights for Imbalanced Classes

We can use `CrossEntropyLoss` in PyTorch, which supports class weights. Here's how you can modify the training script to include class weights:

1. **Determine the Class Distribution:** Calculate the number of samples for each class ("up", "left", "right", "down") in your dataset. This will help you determine the weights.
2. **Set the Class Weights:** Use the inverse of the class counts to assign more weight to the less frequent classes (e.g., "left" and "right"). The weights will be **higher** for classes with **fewer samples**.
3. **Integrate the Weights into `CrossEntropyLoss`:** Use the calculated class weights with `CrossEntropyLoss` to influence the training process.

Example: Adding Class Weights to the Training Script

Here's an example of how you can integrate class weights into your FastAI training process:

```
# Count occurrences of each class                                    Copy code
label_counts = data['key'].value_counts()
total_samples = len(data)

# Compute inverse frequency for each class
weights = torch.tensor([total_samples / label_counts[cls] for cls in dls.vocab], dtype=tor

# Create CrossEntropyLoss with class weights
loss_func = CrossEntropyLoss(weight=weights)

# Define and train the model using ResNet34 with custom loss function
learn = vision_learner(dls, resnet34, pretrained=False, loss_func=loss_func, metrics=[accu

# Fine-tune the model for 20 epochs
learn.fine_tune(20)

# Save the trained model
learn.export('./models/model_classification_fastai_with_weights.pkl')
```

Explanation:

- **Class Weights Calculation**:
  - `label_counts`: This calculates how many times each key appears in the dataset. For example, if "up" appears 500 times and "left" appears 50 times, it tells us how imbalanced the dataset is.
  - **Inverse Frequency**: By using the inverse frequency (`total_samples / label_counts[cls]`), we give a **higher weight** to classes that are **underrepresented** (e.g., "left" and "right"). This encourages the model to focus more on these classes during training.
- `CrossEntropyLoss` **with Weights**:
  - `CrossEntropyLoss(weight=weights)`: This loss function will apply the weights during training, penalizing mistakes for underrepresented classes more than those for overrepresented ones.
- **Training with Class Weights**:
  - Now, when you train the model, it will focus more on learning the underrepresented "left" and "right" actions due to the weighted loss function. This should improve the model's ability to detect turns during gameplay.

Model Optimization

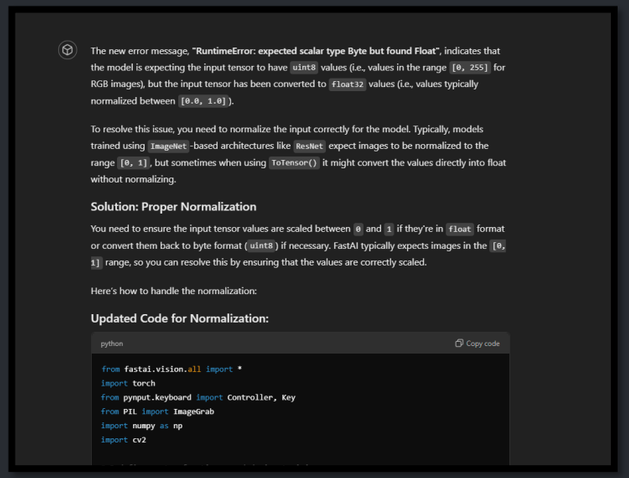
ChatGPT provided guidance on tuning and structures.

Report Drafting

AI assisted in organizing and clarifying complex explanations.

Debugging

AI tools helped resolve preprocessing and training issues.



The new error message, "**RuntimeError: expected scalar type Byte but found Float**", indicates that the model is expecting the input tensor to have `uint8` values (i.e., values in the range `[0, 255]` for RGB images), but the input tensor has been converted to `float32` values (i.e., values typically normalized between `[0.0, 1.0]`).

To resolve this issue, you need to normalize the input correctly for the model. Typically, models trained using `ImageNet`-based architectures like `ResNet` expect images to be normalized to the range `[0, 1]`, but sometimes when using `ToTensor()` it might convert the values directly into float without normalizing.

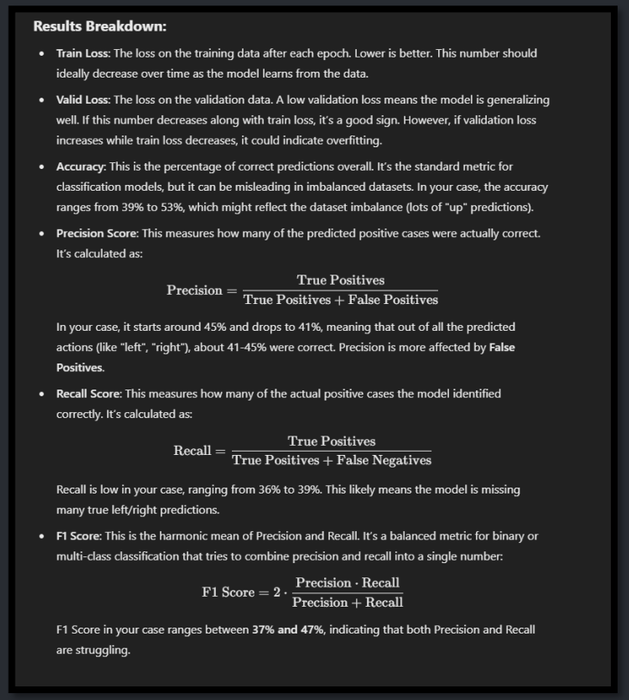
Solution: Proper Normalization

You need to ensure the input tensor values are scaled between `0` and `1` if they're in `float` format or convert them back to byte format (`uint8`) if necessary. FastAI typically expects images in the `[0, 1]` range, so you can resolve this by ensuring that the values are correctly scaled.

Here's how to handle the normalization:

Updated Code for Normalization:

```
python                                                                Copy code
from fastai.vision.all import *
import torch
from pynput.keyboard import Controller, Key
from PIL import ImageGrab
import numpy as np
import cv2
```



Results Breakdown:

- **Train Loss:** The loss on the training data after each epoch. Lower is better. This number should ideally decrease over time as the model learns from the data.
- **Valid Loss:** The loss on the validation data. A low validation loss means the model is generalizing well. If this number decreases along with train loss, it's a good sign. However, if validation loss increases while train loss decreases, it could indicate overfitting.
- **Accuracy:** This is the percentage of correct predictions overall. It's the standard metric for classification models, but it can be misleading in imbalanced datasets. In your case, the accuracy ranges from 39% to 53%, which might reflect the dataset imbalance (lots of "up" predictions).
- **Precision Score:** This measures how many of the predicted positive cases were actually correct. It's calculated as:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

  In your case, it starts around 45% and drops to 41%, meaning that out of all the predicted actions (like "left", "right"), about 41–45% were correct. Precision is more affected by **False Positives**.
- **Recall Score:** This measures how many of the actual positive cases the model identified correctly. It's calculated as:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

  Recall is low in your case, ranging from 36% to 39%. This likely means the model is missing many true left/right predictions.
- **F1 Score:** This is the harmonic mean of Precision and Recall. It's a balanced metric for binary or multi-class classification that tries to combine precision and recall into a single number:

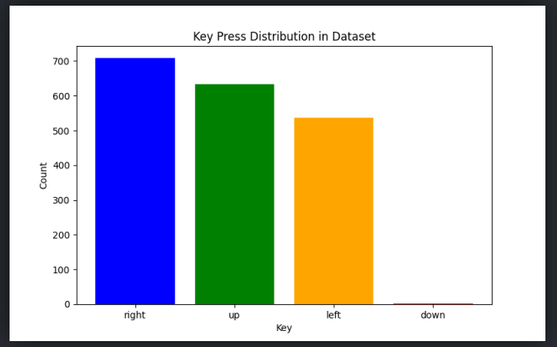
$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

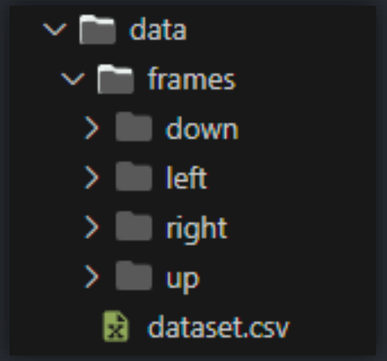
  F1 Score in your case ranges between **37% and 47%**, indicating that both Precision and Recall are struggling.

Conclusion

Data Balance

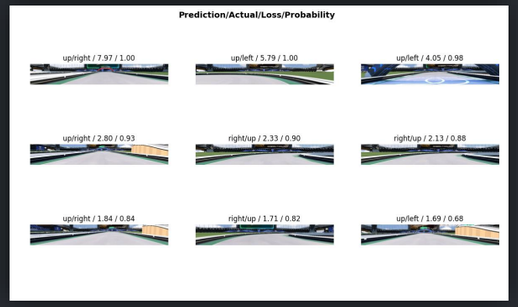
Turn data essential for accurate navigation.





Architecture Limitations

Static CNNs lack temporal and spatial complexity.



Future Enhancements

Temporal models and contextual inputs could improve performance.